UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura

E.P. Ingeniería de Sistemas

Curso: Desarrollo de Aplicaciones Móviles (DAM) · Ciclo VI / 2026-2

ACTIVIDAD AUTÓNOMA N.º 10

VALIDACIÓN DE EXPIRACIÓN, ALMACENAMIENTO SEGURO Y CIERRE DE SESIÓN JWT EN PHARMAMOBIL

Estudiante: Julio Fabián Rodríguez Bazán

Sesión: 10 · Sesión y token

Repositorio Git: https://github.com/fabianrodriguez33/Pharma-Mobil

Rama entregada: https://github.com/fabianrodriguez33/Pharma-Mobil/tree/feature/auth-jwt-rodriguez

Fecha: 19/10/2026

1. Resumen y entorno de verificación

Este informe documenta los cuatro productos exigidos por la Actividad Autónoma N.º 10: la bitácora de seis escenarios de token provocados, la evidencia del almacenamiento seguro, la verificación del cierre de sesión y la matriz de permisos por rol. Todo se ejecutó contra PharmaSoft real.

Elemento	Detalle
App	PharmaMobil (Kotlin Multiplatform + Ktor), rama feature/auth-jwt-rodriguez
Backend	PharmaSoft: Spring Boot + Spring Security JWT, Oracle Free 23ai (Docker, puerto 1522)
Dispositivo	Emulador Android Pixel 8, URL base http://10.0.2.2:8080/
Usuarios semilla	admin / Admin123 · vendedor / Vendedor123 · almacen / Almacen123
Compilación	./gradlew testAndroidHostTest assembleDebug → BUILD SUCCESSFUL (66 pruebas, 0 fallidas)

Nota sobre los mensajes: los textos literales son los que devolvió el backend en esta ejecución. En el escenario 6 el backend responde «No tiene permisos para realizar esta operación» (el enunciado menciona «No tiene permisos suficientes»); se registra el literal real.

Nota sobre el gancho de prueba: para los escenarios 3 y 4 se usó un gancho temporal en AlmacenSeguroAndroid.leerToken() (alterar el último carácter de la firma o devolver null) activado por un archivo en el almacenamiento privado de la app. El gancho no forma parte del código entregado: se revirtió y la APK final se compiló sin él.

2. Producto 1: bitácora de 6 escenarios de token

N.º	Escenario	HTTP	Mensaje del servidor	Reacción de la app
1	Token válido	200	—	Lista el catálogo
2	Token expirado	401	El token ha expirado. Inicie sesión nuevamente	Borra el token y va a Login con aviso
3	Token manipulado	401	El token no es válido	Borra el token y va a Login con aviso
4	Token ausente	401	Debe iniciar sesión para acceder a este recurso	Va a Login con aviso
5	Prefijo incorrecto	401	Debe iniciar sesión para acceder a este recurso	No aplica a la UI (ver ficha)
6a	Sin permiso	403	No tiene permisos para realizar esta operación	Mantiene la sesión y avisa en pantalla
6b	Usuario deshabilitado	401	El usuario está deshabilitado	Borra el token y va a Login con aviso

Escenario 1: Token válido

Campo	Registro
Fecha / hora	2026-10-13 00:11:27 (login) – 00:12:04 (consulta); curl 2026-10-12 23:56:32
Pasos ejecutados	1) Login con admin / Admin123 en la app. 2) Abrir la pestaña Inventario. 3) El plugin Auth adjunta la cabecera y el backend responde.
Código HTTP	200 OK (Logcat: RESPONSE 200; curl: 200)
Mensaje literal (JSON message)	No aplica (respuesta correcta)
Reacción de la UI	El inventario se lista con formulario y acciones de edición (rol admin).
Conclusión técnica	Línea base: con un token vigente y un rol con permiso, el flujo completo (login, almacén, cabecera automática, respuesta) funciona sin código manual de autenticación.

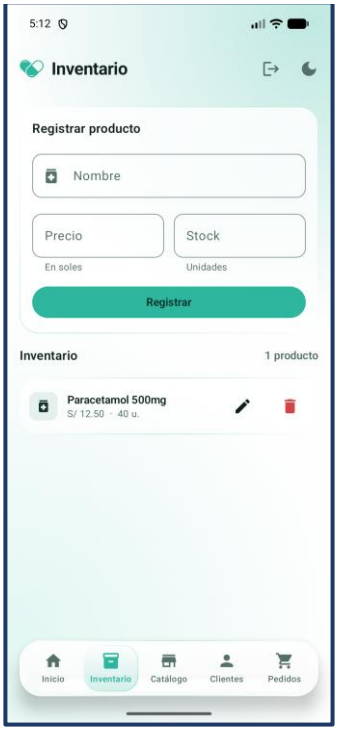


Figura 1. Escenario 1: catálogo cargado con token válido

Escenario 2: Token expirado

Campo	Registro
Fecha / hora	Backend con 1 min: login 2026-10-13 00:08:55; consulta 00:09:50 (+55 s). curl: 2026-10-12 23:57:47 (+75 s)
Pasos ejecutados	1) app.jwt.expiracion-minutos: 1 en application-dev.yaml y reinicio del backend. 2) Login con admin. 3) Esperar más de un minuto. 4) Abrir Inventario para forzar una consulta.
Código HTTP	401 Unauthorized (Logcat: RESPONSE 401)

Mensaje literal (JSON message)	El token ha expirado. Inicie sesión nuevamente
Reacción de la UI	Vuelve a LoginScreen con «Tu sesión expiró. Inicia sesión nuevamente.».
Conclusión técnica	La expiración es un hecho temporal: el servidor lo detecta con la claim exp aunque la firma sea correcta. El cliente no puede saberlo de antemano (no valida la fecha), por eso la detección ocurre en la primera respuesta 401 y no al abrir la app.



Figura 2. Escenario 2: sesión expirada, redirección a Login con aviso

Escenario 3: Token manipulado

Campo	Registro
Fecha / hora	UI 2026-10-13 00:16:53; curl 00:18:07
Pasos ejecutados	1) Login con admin. 2) Con el gancho temporal se altera el último carácter de la firma del token leído del almacén. 3) Abrir Inventario. 4) Revisar el archivo almacen_seguro.xml.
Código HTTP	401 Unauthorized
Mensaje literal (JSON message)	El token no es válido
Reacción de la UI	Vuelve a LoginScreen con aviso; almacen_seguro.xml queda vacío (<map />).
Conclusión técnica	Aquí falla la integridad, no el tiempo: el HMAC no coincide, el mensaje cambia a «no es válido» y el token jamás se podría reutilizar; por eso es indispensable limpiar el almacén y no reintentar con él.

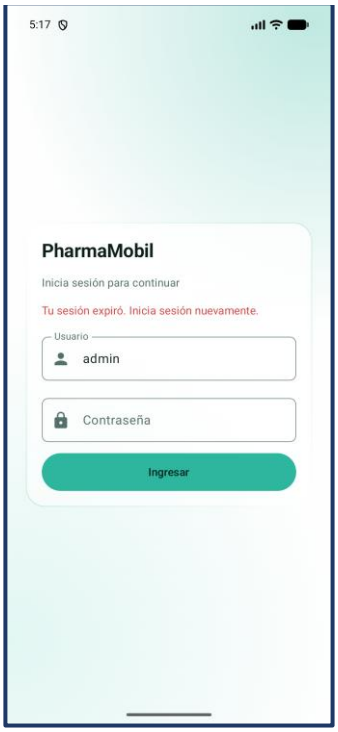


Figura 3. Escenario 3: token manipulado, sesión cerrada

Escenario 4: Token ausente / sin sesión

Campo	Registro
Fecha / hora	UI 2026-10-13 00:17:43; curl 00:18:08
Pasos ejecutados	1) Login con admin. 2) Con el gancho temporal el almacén devuelve null. 3) Abrir Inventario: la petición sale sin cabecera Authorization.
Código HTTP	401 Unauthorized
Mensaje literal (JSON message)	Debe iniciar sesión para acceder a este recurso
Reacción de la UI	Vuelve a LoginScreen con aviso.
Conclusión técnica	Es un fallo de credencial, no de token: el filtro ni siquiera intenta validar nada y responde con el mensaje genérico de autenticación requerida; muestra que la guardia de la UI y la del servidor son independientes y se refuerzan.

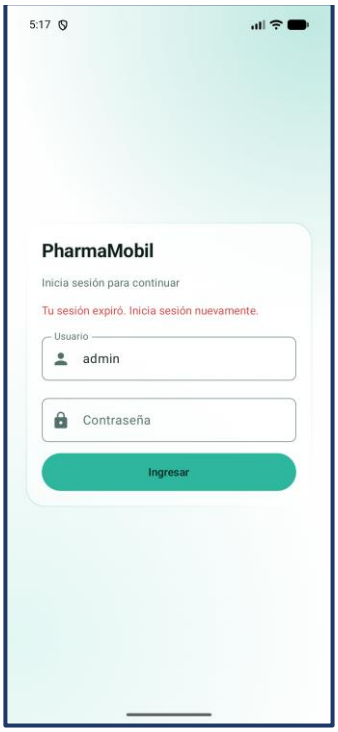


Figura 4. Escenario 4: petición sin token, redirección a Login

Escenario 5: Prefijo incorrecto o mal formado

Campo	Registro
Fecha / hora	curl 2026-10-13 00:18:08
Pasos ejecutados	1) Obtener un token válido. 2) Enviar «authorization: bearer <token>» (minúscula), «Bearer » sin token, el token sin prefijo y el esquema Basic.
Código HTTP	401 Unauthorized (las cuatro variantes)
Mensaje literal (JSON message)	Debe iniciar sesión para acceder a este recurso
Reacción de la UI	No se reproduce en la UI: el plugin Auth siempre envía «Bearer »; la prueba es a nivel HTTP.
Conclusión técnica	El filtro solo reconoce el esquema «Bearer » exacto; cualquier variante equivale a no enviar credencial. Delegar la cabecera al plugin Auth evita este error de raíz, que sí es frecuente al escribirla a mano.

```
curl contra PharmaSoft - bitacora y matriz

== S1 token valido 2026-10-12 23:56:32
GET /productos con token recien emitido: 200
== S2 token expirado (app.jwt.expiracion-minutos=1) 2026-10-12 23:57:47
GET /productos 75 s despues del login: 401 | El token ha expirado. Inicie sesión nuevamente
== S3 token manipulado 2026-10-13 00:18:07
401 | El token no es válido
== S4 sin cabecera 2026-10-13 00:18:08
401 | Debe iniciar sesión para acceder a este recurso
== S5 prefijo incorrecto 2026-10-13 00:18:08
bearer minuscula: 401 | Debe iniciar sesión para acceder a este recurso
'Bearer' sin token: 401 | Debe iniciar sesión para acceder a este recurso
sin prefijo: 401 | Debe iniciar sesión para acceder a este recurso
Basic: 401 | Debe iniciar sesión para acceder a este recurso
== S6b vendedor sin permiso 2026-10-13 00:18:11
POST producto vendedor: 403 | No tiene permisos para realizar esta operación
DELETE producto vendedor: 403 | No tiene permisos para realizar esta operación
== Matriz 2026-10-13 00:18:12
Listar productos: admin=200 vendedor=200 almacen=200
Registrar productos: admin=201 vendedor=403 almacen=409
Listar clientes: admin=200 vendedor=200 almacen=403
Registrar venta (cuerpo vacio): admin=400 vendedor=400 almacen=403
Ver reportes: admin=200 vendedor=403 almacen=403
Consultar /auth/me: admin=200 vendedor=200 almacen=200
almacen registrar (nombre distinto): 201
curl con token previo:
    {"timestamp":"2026-10-
    13T00:20:43.404342500","status":401,"error":"Unauthorized","message":"El usuario está
    deshabilitado","path":"/api/v1/productos","validationErrors":null} 401
```

Figura 5. Escenarios 5 y 6: salida de curl contra PharmaSoft (bitácora y matriz)

Escenario 6a: Acción sin permiso (403)

Campo	Registro
Fecha / hora	UI 2026-10-13 00:20:28; curl con vendedor 00:18:11
Pasos ejecutados	1) curl con vendedor: POST y DELETE de producto. 2) En la UI el vendedor no tiene formulario (acciones ocultas). 3) Con una sesión de almacen abierta se le cambió el rol a VENDEDOR en la base y se intentó registrar un producto.
Código HTTP	403 Forbidden
Mensaje literal (JSON message)	No tiene permisos para realizar esta operación
Reacción de la UI	La sesión sigue activa; se muestra «No tienes permiso para realizar esta acción» sobre la misma pantalla.
Conclusión técnica	Autenticado no es autorizado: el 403 indica que la identidad es válida pero el permiso no alcanza, por eso la sesión se conserva; cerrarla castigaría al usuario por un error de permisos y la UI ya debería haber ocultado la acción.

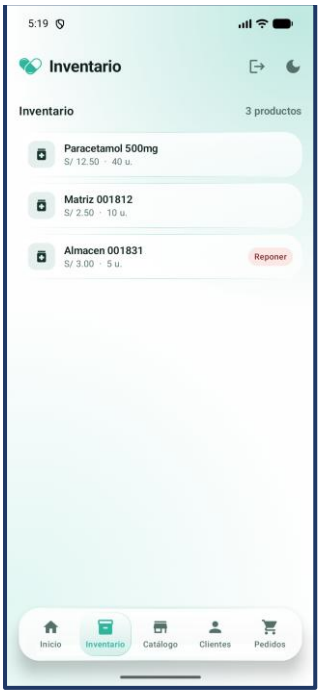


Figura 6a. vendedor: inventario sin formulario ni acciones



Figura 6b. 403 con la sesión abierta

Escenario 6b: Usuario deshabilitado

Campo	Registro
Fecha / hora	UI 2026-10-13 00:20:43; curl 00:20:43
Pasos ejecutados	1) Sesión de almacen abierta. 2) UPDATE usuarios SET estado = 0. 3) Abrir Catálogo (nueva consulta). 4) Se restauró estado = 1.
Código HTTP	401 Unauthorized
Mensaje literal (JSON message)	El usuario está deshabilitado
Reacción de la UI	Vuelve a LoginScreen con aviso.
Conclusión técnica	El JWT no es suficiente por sí solo: el backend consulta el estado del usuario en cada petición, de modo que un administrador puede revocar el acceso al instante sin esperar a que el token expire.



Figura 7. Escenario 6b: usuario deshabilitado, sesión cerrada

3. Producto 2: evidencia del almacenamiento seguro

Implementación: AlmacenSeguroAndroid cifra con AES-256 en modo GCM usando una clave del Android Keystore (alias pharmamobil_token_key, no exportable). El IV de 12 bytes viaja antepuesto al texto cifrado y el resultado se guarda en Base64 en SharedPreferences (archivo almacen_seguro.xml). Los nombres de clave también se ofuscan con SHA-256. Se implementó con javax.crypto/Keystore y no con EncryptedSharedPreferences de Jetpack.

Comprobación 1: almacén cifrado en disco

Contenido real de shared_prefs/almacen_seguro.xml (leído con adb run-as, equivalente al Device Explorer). Ni el token ni el perfil aparecen en texto plano y los nombres de clave son resúmenes SHA-256.

```
adb shell run-as pe.edu.upeu.pharmamobil cat shared_prefs/almacen_seguro.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="b6fcc69f7bd0d90a454bc88b922168f4473c10cfc7 ... [cifrado AES-GCM, 421 caracteres Base64] ...
    w==</string>
    <string name="3c469e9d6c5875d37a43f353d4f88e61fcf812c66e ... [cifrado AES-GCM, 669 caracteres Base64] ...
    Ws=</string>
</map>
```

Figura 8. almacen_seguro.xml: valores cifrados (AES-GCM) y claves ofuscadas

Comprobación 2: registro con cabecera saneada

Con Ktor Logging en nivel HEADERS y sanitizeHeader sobre Authorization, Logcat imprime «Authorization: ***». El token nunca se escribe en el registro (el formato impreso es *** en lugar de «Bearer ***»).

```
adb logcat (System.out) - Ktor Logging nivel HEADERS
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos?pagina=0&tamanio=20
METHOD: GET
COMMON HEADERS
-> Accept: application/json
-> Authorization: ***
-> Content-Type: application/json
CONTENT HEADERS
-> Content-Length: 0
HttpClient: RESPONSE: 200
METHOD: GET
FROM: http://10.0.2.2:8080/api/v1/productos?pagina=0&tamanio=20
COMMON HEADERS
-> content-type: application/json
```

*Figura 9. Logcat: Authorization: *** en la petición autenticada*

Comprobación 3: persistencia tras reinicio

Se inició sesión, se mató el proceso (am force-stop) y se reabrió la app: entra directamente a la aplicación sin pedir login (2026-10-13 00:12:20).

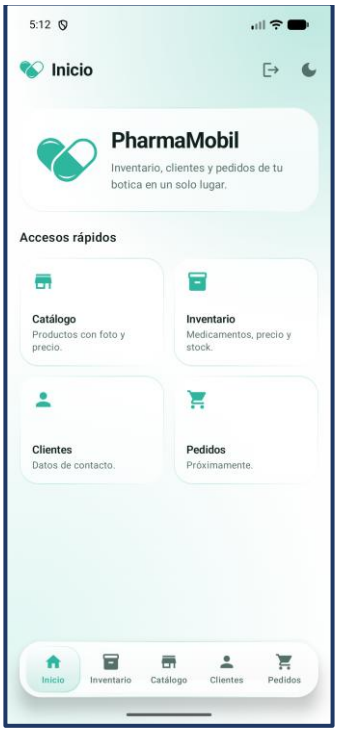


Figura 10. La app reabre con la sesión restaurada desde el almacén cifrado

4. Producto 3: cierre de sesión completo

Efecto	Implementación	Evidencia
1. Borrado físico del token	AuthRepositoryRest.cerrarSesion() → almacen.borrarToken()	Figura 11: almacen_seguro.xml pasa a <map />; prueba CerrarSesionTest
2. Estado en memoria a null	_sesion.value = null (StateFlow sesionActual)	CerrarSesionTest: sesionActual y obtenerSesion() == null
3. Redirección a LoginScreen	App.kt: sesion == null → LoginScreen (SesionViewModel.salir())	Figuras 12 y 13: Login inmediato y tras reiniciar

4. Invalidar caché del plugin Auth	client.limpiarTokenEnCache() → authProvider<BearerAuthProvider>()?.clearToken()	CerrarSesionTest: la petición posterior sale sin cabecera Authorization

almacen_seguro.xml antes y despues de cerrarSesion()

```
--- ANTES de cerrarSesion (2026-10-13 00:21:53)
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="b6fcc69f7bd0d90a454bc88b922168f4473c10cfc713978dec404fbb90fbe25a">nLMxiii/+QFcwCv0GKcOB61kNl
    <string name="3c469e9d6c5875d37a43f353d4f88e61fcf812c66eee3457465a40b0da4153e0">Gj46ZIfEx4vkKF/dKRAfNgAmB0
</map>
--- DESPUES de cerrarSesion (2026-10-13 00:22:04)
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map />
```

Figura 11. Efecto 1: almacén antes y después de cerrarSesion()

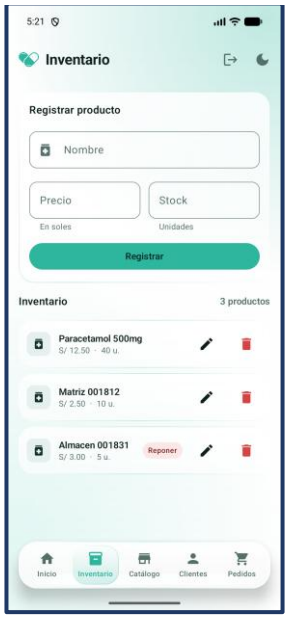


Figura 12a. Antes: sesión activa



Figura 12b. Después: LoginScreen



Figura 13. Reinicio: sigue en Login

La prueba automatizada usa un MockEngine: inicia sesión, verifica que la petición lleva «Bearer jwt-de-prueba», llama a cerrarSesion() y comprueba que el almacén queda vacío, la sesión es null y la siguiente petición ya no envía Authorization (la caché del plugin se invalidó).

gradlew testAndroidHostTest - CerrarSesionTest

```
$ ./gradlew testAndroidHostTest assembleDebug
PASSED  CerrarSesionTest.cerrarSesionAplicaLosCuatroEfectos
PASSED  LoginViewModelTest.loginCorrectoTerminaEnExitoYLimpiaLaClave
PASSED  LoginViewModelTest.credencialesInvalidasMuestranMensajeEspecifico
PASSED  LoginViewModelTest.camposVaciosNoLlamanAlRepositorio
PASSED  LoginViewModelTest.sesionPuedeSoloConPermisosConcedidos

Pruebas ejecutadas: 66   fallidas: 0
BUILD SUCCESSFUL
```

Figura 14. Resultado de las pruebas: CerrarSesionTest y LoginViewModelTest en verde

5. Producto 4: matriz de permisos por rol

Resultados reales de 6 operaciones × 3 usuarios (HTTP contra PharmaSoft, 2026-10-13 00:18) y lo que ofrece la UI de PharmaMobil.

Operación	admin	vendedor	almacen
Listar productos	200 OK (UI completa)	200 OK (solo lectura)	200 OK (lectura/escritura)
Registrar/Editar productos	201 Created	403 Forbidden (formulario oculto)	201 Created* (formulario visible)
Listar clientes	200 OK	200 OK	403 Forbidden (pestaña oculta)
Registrar venta**	400 (autorizado)	400 (autorizado)	403 Forbidden (sección oculta)
Ver reportes	200 OK	403 Forbidden	403 Forbidden
Consultar /auth/me	200 OK	200 OK	200 OK

* En la ejecución de la matriz el primer intento de almacen devolvió 409 porque el nombre ya existía (autorizado); con un nombre distinto devolvió 201. ** Cuerpo vacío: 400 de validación indica permiso concedido; 403, denegado. La app aún no tiene módulo de ventas ni de reportes, por lo que esas filas se verificaron por HTTP.

Configuración restaurada

- app.jwt.expiracion-minutos: 60 en application-dev.yaml de PharmaSoft (backend reiniciado).
- Usuario almacen con estado = true y rol ALMACENERO tras las pruebas de deshabilitado y de cambio de rol.
- README.md: sección «Seguridad y Gestión de Tokens JWT» agregada en el repositorio.

6. Limitaciones

- iOS: AlmacenSeguroIos (Keychain) está implementado pero no se compiló ni probó (requiere Mac con Xcode).
- Los escenarios 3 y 4 en la UI requirieron un gancho temporal de prueba (no incluido en el código entregado).
- El escenario 5 se evidencia a nivel HTTP: la app no puede emitir un prefijo distinto de «Bearer».
- El aviso 403 se muestra como texto en la pantalla del formulario, no como Snackbar.
- La implementación usa Keystore + AES-GCM directos; no emplea EncryptedSharedPreferences.